Téma 2: Čistenie dát

📖 Čo sa tu naučím

- Parsovať Apache Combined Log Format regexom (a prečo nie `pd.read_csv`)
- Aké HTTP metódy / status kódy / URL filtrovať a prečo
- Identifikovať robotov 3 spôsobmi: `robots.txt`, User-Agent, IP zoznam
- Spojiť 4 boolean filtre v pandas a kontrolovať počty po každom kroku
- Konkrétne počty: 257k → 57 650 → ~31k → 333 IP → ~20 666 záznamov

🔑 Najdôležitejšie vzorce a koncepty

> **Z prednášky 6 (Fázy procesu Web Log Miningu — CRISP-DM II):** Proces objavovania znalostí na základe používania webu sa rozdeľuje do troch základných častí. Prvou fázou je **príprava dát**, ktorá obsahuje postupnosť krokov: získanie súboru z webového servera → **čistenie súboru od nepotrebných dát** → **čistenie súboru od robotov vyhľadávacích služieb** → identifikácia používateľov na základe rôznej verzie internetového prehliadača → identifikácia sedení na základe času/štruktúry → doplnenie ciest. Téma 2 (čistenie dát) pokrýva práve **prvé dva kroky** tejto pipeline — bez nich sa nedá objektívne identifikovať používateľ ani sedenie, lebo vstupom by bol skreslený šum (statické súbory, chybové odpovede, automatizovaní agenti).

Apache Combined Log Format (1 riadok):

```
IP - - [DateTime] "Method URL HTTP/Version" StatusCode Bytes "Referrer" "Agent"
```

Pandas boolean masking (idiom celej témy):

```
df = df[~df["Agent"].str.contains(r"bot|crawl|spider", case=False, na=False)]
```

- `~` = negácia masky (ponechať NON-match)
- `na=False` = NaN neráta ako match (inak NaN propaguje cez `~`)
- `case=False` = case-insensitive
- regex alternácia `a|b|c` = OR cez kľúčové slová

Status kódy: - `2xx` úspech | `3xx` presmerovanie | `4xx` chyba klienta | `5xx` chyba servera - Čistenie I: `[200, 399]` (úspech + presmerovanie) - Čistenie II: `[200, 299]` (len reálne videná stránka)

Heuristika `robots.txt`: každá IP čítajúca `/robots.txt` ≈ robot (bežný používateľ na tento súbor nemá link).

Úloha 01: parsovanie_a_cistenie.py (Čistenie I)

Teoretický základ: Podľa metodiky CRISP-DM (fáza prípravy dát) je prvým krokom **čistenie súboru od nepotrebných dát** — t.j. odstránenie záznamov, ktoré nereprezentujú reálne navštívenie stránky používateľom. Apache log je **pasívny záznam** server-side: zachytí každý HTTP request, vrátane stiahnutia statických prostriedkov (CSS, JS, obrázky), chybových stavov a iných metód než `GET`. Práve preto čistenie I filtruje na úroveň "len úspešne doručené HTML/dynamické stránky". *(viď prednáška 6 — Fázy procesu, krok "Čistenie súboru od nepotrebných dát").*

Čo robí: Regex-parsuje surový Apache log do DataFrame a postupne filtruje podľa metódy, status kódu a prípon URL.

Vstup → výstup: - Vstup: `in/Log_jeden_den.log` (~257 000 riadkov) - Výstup: `out/05_cleaned_log.csv` (~57 650 riadkov, stĺpce: IP, DateTime, RequestMethod, URL, StatusCode, Referrer, Agent)

Kľúčové kroky: 1. `re.compile(pattern)` — jeden regex pre celý log 2. Stream-parse riadok po riadku (O(1) pamäť), `pattern.match(line)` 3. Filter `RequestMethod == "GET"` 4. Filter `StatusCode.between(200, 399)` 5. Filter `~URL.str.contains(unwanted_ext)` — odstránenie statických súborov 6. `reset_index(drop=True)` + `to_csv(index=False)`

Dôležité parametre: - `errors="replace"` — poškodené UTF-8 bajty nahradí U+FFFD (boti posielajú binárny smetok) - `pattern.match()` (nie `search()`) — kotví regex na začiatok riadku (rýchlejšie) - `(?:\?|$)` v regexe statík — prípona musí byť pred `?` alebo na konci URL (aby `.cssfoo` nematchol) - `re.IGNORECASE` — `.JPG` == `.jpg` - Cookie, user, Bytes sa zámerne **NEUKLADAJÚ** (zadanie)

Vzorce / formulácie:

```
# Apache regex (skupiny 1..8):
^(\S+)\s+\S+\s+\S+\s+         # 1=IP, preskočené Cookie+user
\[([^\]]+)\]\s+                # 2=DateTime
"(\S+)\s+(\S+)\s*\S*"\s+       # 3=Method, 4=URL, verzia neulož
(\d{3})\s+                     # 5=Status
(\S+)\s+                       # 6=Bytes (zachytené, neulož)
"([^"]*)"\s+"([^"]*)"          # 7=Referrer, 8=Agent

# Statické prípony:
\.(?:bmp|jpe?g|png|gif|css|flv|ico|swf|rss|xml|cur|js|json|svg|woff2?|eot|ttf|otf)(
```

Časté otázky pri obhajobe: - **Q:** Prečo nie `pd.read_csv`? **A:** Apache log nie je CSV — má `"..."` so vnorenými medzerami, `[...]` na timestamp, premenlivé separátory. - **Q:** Prečo `between(200, 399)` a nie `(200, 299)`? **A:** Tu chceme zachovať aj 3xx (presmerovanie 301/302/304) — používateľ dostal odpoveď. Filter na 2xx príde v Čistení II. - **Q:** Prečo výstup 57 650, nie 56 500 (zadanie)? **A:** ~2 % odchýlka — chýbajú prípony `.map`, `.txt`, `.pdf`; "približne" v zadaní toleruje. - **Q:** Kam tento krok zapadá v CRISP-DM? **A:** Do fázy **prípravy dát**, konkrétne podkrok "čistenie súboru od nepotrebných dát" — prvý filter pred odstránením robotov *(viď prednáška 6).*

Úloha 02: dalsie_cistenie.py (Čistenie II)

Teoretický základ: Druhý podkrok prípravy dát podľa CRISP-DM je **čistenie súboru od robotov vyhľadávacích služieb**. Roboti (crawlery vyhľadávačov) navštevujú stránky automaticky, často s neľudskou rýchlosťou a šírkou — bez ich odstránenia by všetky následné metriky (dĺžka sedenia, navigačné vzory) boli skreslené. Tento krok kombinuje detekciu cez **User-Agent** s odstránením duplicitných záznamov a zachováva len reálne zobrazené stránky (status 2xx). *(viď prednáška 6 — sekcia "Čistenie súboru od robotov vyhľadávacích služieb").*

Čo robí: Hlbšie čistenie nad výstupom z Úlohy 01 — boti podľa User-Agent, robots.txt, duplikáty, ďalšie URL vzory, len 2xx.

Vstup → výstup: - Vstup: `in/05_cleaned_log.csv` (~57 000) - Výstup: `out/06_cleaned_log.csv` (~31 500)

Kľúčové kroky: 1. Odstrániť boty: `Agent ~ bot|crawl|spider|slurp|archiver|curl|wget|libwww|perl|ltx71|facebook` 2. Odstrániť `robots.txt` v URL 3. `drop_duplicates(subset=["IP","DateTime","URL"])` — F5 / dvojklik 4. Odstrániť `image?view=image`, `.pdf`, `/component/` (Joomla), `format=feed|/feed` 5. `between(200, 299)` — len reálne zobrazené stránky 6. `reset_index(drop=True)` + `to_csv(index=False)`

Dôležité parametre: - `na=False` v `str.contains` — chráni pred NaN propagáciou cez negáciu - `case=False` — `Robots.txt` == `robots.txt` - `drop_duplicates(subset=[...], keep="first")` — porovnáva len dané stĺpce, nie celý riadok - `between(200, 299)` je **inkluzívne na oboch koncoch**

Vzorec / formula:

```
bot_pattern = re.compile(
    r'bot|crawl|spider|slurp|archiver|curl|wget|libwww|perl|ltx71|facebook',
    re.IGNORECASE)
df = df[~df["Agent"].str.contains(bot_pattern, na=False)]
```

Časté otázky pri obhajobe: - **Q:** Prečo dropujeme aj 3xx tu, keď v Čistení I sme ich nechali? **A:** 3xx je presmerovanie — cieľová stránka má vlastný 2xx záznam, inak by sme rátali dvojnásobne. - **Q:** Prečo `drop_duplicates(IP, DateTime, URL)` a nie celý riadok? **A:** Status/Referrer sa medzi duplikátmi líšiť nemajú; tieto 3 stĺpce stačia na identifikáciu "rovnaký prístup v rovnakom čase z rovnakej IP". - **Q:** Prečo `\.pdf` s escape? **A:** Bodka v regexe je metaznak (akýkoľvek znak); escape `\.` = doslovná bodka, aby sa `apdf` nematchol.

Úloha 03: identifikacia_robot_ip.py (Čistenie III)

Teoretický základ: Druhým spôsobom detekcie robota (popri User-Agent) je **prístup k súboru** `robots.txt` v koreni domény. Tento súbor je definovaný Robots Exclusion Protocolom (RFC 9309) a slúži ako inštrukčný zoznam pre crawlerov — bežný používateľ ho nikdy nestiahne, lebo nemá zo žiadnej stránky link naň. Preto sa každá IP adresa, ktorá si o `robots.txt` požiadala, považuje za vysoko pravdepodobne robotskú. Výstupom je zoznam IP, ktorý použije Úloha 04 ako druhý filter. *(viď prednáška 6 — krok "Čistenie súboru od robotov vyhľadávacích služieb").*

Čo robí: Identifikuje IP adresy robotov na základe prístupu k `/robots.txt` v surovom logu.

Vstup → výstup: - Vstup: `in/Log_jeden_den.log` (surový log!) - Výstup: `out/07_robot_ips.csv` (1 stĺpec `IP`, **333 záznamov** vrátane duplicít, ~108 unikátnych)

Kľúčové kroky: 1. Použiť **rovnaký regex** ako v Úlohe 01 (konzistentnosť pipeline) 2. Stream-parse log, pre každý match: ak `"robots.txt" in url.lower()` → pridať IP 3. Uložiť ako CSV s hlavičkou `IP` (duplikáty zachované)

Dôležité parametre: - `url.lower()` + substring `in` — case-insensitive heuristika - Duplikáty sa **nezahadzujú** — zadanie ráta 333 záznamov, nie 333 unikátnych IP - Stream parsing (`for line in f`) — log môže mať GB

Teória — prečo robots.txt: - `robots.txt` je súbor v koreni domény (Robots Exclusion Protocol, RFC 9309) - Číta ho **iba crawler** pred indexáciou - Bežný browser ho nikdy nestiahne — neexistuje naňho link - → Heuristika: každá IP žiadajúca `/robots.txt` ≈ bot

Časté otázky pri obhajobe: - **Q:** Prečo nedrop_duplicates? **A:** Zadanie hovorí "333 IP adries" — interpretované ako 333 záznamov, nie 333 unikátnych. Unikátnych je ~108. - **Q:** Kde je slabosť substring `in`? **A:** Matchne aj `robots.txt.bak` alebo `foo-robots.txtdownload`. Bezpečnejšie `url.lower().endswith("/robots.txt")`. - **Q:** Prečo vlastný regex namiesto pd.read_csv (zadanie ho odporúča)? **A:** Apache log nie je CSV (medzery v `"..."`, `[...]`).

Úloha 04: filtrovanie_robotov.py (Čistenie IV)

Teoretický základ: Finálne čistenie spája všetky známe spôsoby detekcie robota do jedného OR filtra: **User-Agent kľúčové slová** (priznaný bot), **automatizované nástroje** (curl/wget/cron —

nie sú prehliadače), **prázdny User-Agent** (`"-"` — pravdepodobne skript) a **IP zoznam** z Úlohy 03 (prístup k `robots.txt`). Až po tomto kroku má dataset reprezentovať **reálnych ľudských používateľov** a môže ísť do nasledujúcej fázy prípravy dát — identifikácie používateľov a sedení. *(viď prednáška 6 — prechod z "Bez robotov" do "Rozlíšenie sedení na základe agenta").*

Čo robí: Finálne odstránenie robotov z logu — kombinuje User-Agent kľúčové slová, prázdny UA a zoznam IP z Úlohy 03.

Vstup → výstup: - Vstup: `in/05_cleaned_log.csv` (výstup Úlohy 01) + `in/07_robot_ips.csv` (Úloha 03) - Výstup: `out/08_cleaned_log.csv` (~20 666 riadkov) — vstup pre cv2

Kľúčové kroky: 1. Boti UA: `r"bot|crawl|spider"` (case=False, na=False) 2. Automatizované: `r"cron|curl|wget|libwww|ltx71"` 3. Prázdny UA: `df = df[df["Agent"] != "-"]` (presná rovnosť, nie contains!) 4. Robot IP: `df = df[~df["IP"].isin(robot_ips)]` 5. `reset_index(drop=True)` + `to_csv(index=False)`

Dôležité parametre: - `pd.read_csv("...")["IP"].unique()` — `.unique()` len optimalizuje `.isin()` lookup - `== "-"` namiesto `contains("-")` — chceme presne pomlčku, nie každý string s pomlčkou - Logika **OR** medzi filtrami: riadok sa zmaže ak match UA **alebo** match IP

Vzorec — kombinovaná filtrácia:

```
bot  = df["Agent"].str.contains(r"bot|crawl|spider", case=False, na=False)
auto = df["Agent"].str.contains(r"cron|curl|wget|libwww|ltx71", case=False, na=Fals
df = df[~bot]
df = df[~auto]
df = df[df["Agent"] != "-"]
df = df[~df["IP"].isin(robot_ips)]
```

Časté otázky pri obhajobe: - **Q:** Prečo UA aj IP filter (nestačí jeden)? **A:** UA sa dá fakeovať (HTTP header je textový), IP nie tak ľahko. Kombinácia = robustnosť. - **Q:** Prečo berieme `05_cleaned_log.csv` , nie `06_` ? **A:** Úloha 06 je paralelná vetva (čistenie cez referrer/obrázky); úloha 08 čistí robotov nad výstupom úlohy 05 priamo. - **Q:** Riziko false-positive pre `cron` ? **A:** Substring bez `\b` (word boundary) — minimálne riziko v praxi. - **Q:** Prečo je tento výstup vstupom pre cv2? **A:** Lebo až po odstránení robotov má zmysel identifikovať používateľov a sedenia — inak by sedenia obsahovali aj crawlery, ktoré nereprezentujú ľudskú navigáciu *(viď prednáška 6 — postupnosť fáz prípravy dát).*

🎯 Vyhodnotenie / na čo si dať pozor pri obhajobe

- **Konkrétne počty vie naspamäť:** 257k → 57 650 → ~31 500 → 333 IP → ~20 666
- **Vie vysvetliť každý** `case=False, na=False` — case-insensitive + NaN chránenie pred propagáciou cez `~`

- **Pozná rozdiel** `between(200,399)` **vs** `between(200,299)` — 3xx ostávajú v I (server odpovedal), padajú v II (dvojité započítanie)
- **Vie obhájiť** `robots.txt` **heuristiku** — Robots Exclusion Protocol, žiadny link, bežný používateľ nestiahne
- **Vie vysvetliť, prečo regex namiesto** `pd.read_csv` — medzery vnútri `"..."`, `[...]`
- **Pozná tri spôsoby detekcie robota:** robots.txt prístup, User-Agent kľúčové slová, IP zoznam (kombinácia OR)
- **Pozná idiom** `reset_index(drop=True)` — diery v indexe po filtroch, downstream skripty predpokladajú 0..N-1
- **Vie zaradiť čistenie do CRISP-DM** — fáza prípravy dát, kroky "čistenie od nepotrebných dát" a "čistenie od robotov" *(viď prednáška 6).*